Final Engineering Summary

Executive Summary

This project delivers a production-grade URL shortener prototype, meant to demonstrate engineering judgment across system design, implementation, testing, and an AI-assisted development workflow. The system handles URL creation, redirect resolution, click analytics, and management, running as containerized services orchestrated with Docker Compose.

There are deliberate choices at every layer: 302 redirects for analytics completeness, Redis counter batching for code-generation efficiency, Kafka for decoupled analytics recording, Bucket4j for rate limiting, and a fail-open AI safety check on the write path. Each choice is documented with rationale in `ARCHITECTURE.md` and `RISKS_AND_TRADEOFFS.md`.

Artifacts Delivered

Backend (Java 17, Spring Boot 3.2.5), 39 source/test files under `backend/src`: entities (`ShortUrl`, `ClickEvent`), repositories, `ShortCodeGenerator`, the four-service split (`WriteService`, `ReadService`, `AnalyticsService`, `UrlInfoService`), the Kafka producer/consumer, REST controllers (`UrlController`, `RedirectController`, `AnalyticsController`), DTOs, custom exceptions plus `GlobalExceptionHandler`, `RateLimitConfig`, `KafkaConfig`, `WebConfig`, `UrlSafetyService` (Claude API integration), unit tests, and the `UrlControllerIT` integration test suite described below.

Frontend (React 18, TypeScript, Tailwind), 7 source files under `frontend/src`: routed pages (Home, Manage, Analytics), the Axios API client, and Tailwind styling.

Infrastructure. `docker-compose.yml` (PostgreSQL, Redis, Zookeeper, Kafka, app, frontend), multi-stage Dockerfiles for both services, an Nginx config for the frontend.

Documentation. This document plus `ARCHITECTURE.md`, `SCENARIO_1_GREENFIELD.md`, `SCENARIO_2_BROWNFIELD.md`, `SCENARIO_3_AMBIGUOUS.md`, `AI_WORKFLOW.md`, `RISKS_AND_TRADEOFFS.md`, and the project-level `CLAUDE.md` (AI custom-instructions/conventions file). Markdown sources live in `docs/`, and PDF renders live at the repo root as the reviewable deliverable set.

Architecture Rationale

PostgreSQL. Relational integrity matters for URL mappings, and the UNIQUE constraints on `code` and `custom_alias` are the critical safety net against race conditions. Full JPQL support also makes analytics aggregation straightforward.

Redis. Two distinct uses: an atomic counter for code generation, where `INCR` enables lock-free distributed allocation, and a cache-aside store for lookups, which cuts PostgreSQL read load on the hot redirect path.

Kafka. Decouples click recording from the redirect response. This is the right architecture for high-volume event streaming where analytics can be eventually consistent, and 7-day retention means events survive consumer downtime without loss.

Spring Boot 3. A standard enterprise Java framework whose auto-configuration minimizes integration boilerplate for JPA, Redis, and Kafka.

React + Vite + Tailwind. A modern frontend stack. Vite's dev proxy avoids CORS friction locally, and Recharts renders the click-over-time chart.

Separation of concerns. The service layer splits by read/write responsibility, not by entity: `WriteService` (create/delete), `ReadService` (redirect resolution, optimized for the hot path), `UrlInfoService` (management UI

reads), `AnalyticsService` (aggregation reads). Each is independently testable, and the read/write performance profile of each stays explicit instead of implicit.

AI Assistance Approach and Outcomes

Workflow: (1) I design the class structure and method contracts, (2) AI generates an implementation from a precise specification, (3) I review it for correctness, style, and edge cases, (4) I accept, edit, or reject it, (5) I write the final design rationale. Full traceability, including the debugging example below, is in `AI_WORKFLOW.md`.

Rough split: AI generated about 70% of code lines, boilerplate, DTOs, entities, repositories, controllers, standard patterns. I wrote the remaining 30% of lines, plus 100% of the design decisions (service split, Redis strategy, Kafka failure handling, rate-limit scope) and 100% of the build/test-infrastructure debugging (the Failsafe wiring and Kafka test-timeout fixes below).

Key rejections: a "god class" service, replaced by the 4-service split; MapStruct, replaced by inline mapping methods; a custom `RedisConfig`, replaced by Spring auto-configuration.

Build and test infrastructure. `UrlControllerIT`'s 4 integration tests were initially never executing (`pom.xml` was missing the `maven-failsafe-plugin` binding), and once fixed, the newly-running suite took 52 seconds because the test profile was attempting real Kafka broker connections. Both are fixed, verified (14/14 tests, 20 seconds), and fully traced in `AI_WORKFLOW.md` and `RISKS_AND_TRADEOFFS.md`. `npm audit` on the frontend surfaced 2 moderate CVEs that both require breaking major-version bumps to fix; the call was to document and defer rather than force an unreviewed upgrade.

Assumptions Made

1. Single-tenant. All users share the URL namespace, no per-user ownership.
2. No authentication. The API is public, and rate limiting is the sole abuse protection.
3. Read:write ratio around 100:1. The Redis cache sizing (24h TTL) assumes this.
4. Analytics are eventually consistent. Sub-second Kafka lag is acceptable.
5. Short codes are 6+ characters once the counter exceeds 62^5, about 916 million.
6. Single geographic region. No CDN, no edge caching, no global distribution.
7. Trusted internal network. `X-Forwarded-For` is trusted without further validation, which is fine behind an internal reverse proxy but not appropriate for direct public internet exposure.

Limitations of the Prototype vs. Production

Area	Prototype	Production
Authentication	None	OAuth2/JWT/SAML integration
Authorization	None	Per-URL ownership, admin roles
Database HA	Single instance	Replication + automated failover
Redis HA	Single instance	Sentinel or Cluster
Rate limiting	In-memory per instance	Redis-backed, shared across instances
Schema migrations	Hibernate auto-DDL	Flyway versioned migrations

Area	Prototype	Production
Secret management	Plaintext in YAML	Vault / Kubernetes Secrets / AWS Secrets Manager
Observability	Actuator health/metrics	Distributed tracing, structured logging, alerting
CDN	None	CloudFront/Fastly for redirect caching
CI/CD	None	Automated pipeline, container registry, rolling deploys
Load testing	None	k6/Gatling throughput benchmarks
GDPR compliance	None	Data retention policy, right-to-erasure on click events
Test execution hygiene	Manual verification (dead IT tests found and fixed)	A CI gate that fails the build if the executed test count regresses
Dependency scanning	`npm audit` run manually, findings deferred with rationale	Automated CVE scanning in CI, gated merges on new highs

What Would Be Done Differently at Production Scale

1. **Authentication first.** Enterprise identity integration (SAML/OAuth2) before anything else. No public APIs.
2. **Flyway migrations.** `ddl-auto: update` isn't acceptable in a regulated environment. Every schema change would become a versioned, reviewed migration.
3. **Redis-backed rate limiting.** Per-IP limits need to be consistent across instances. Single-instance limiting is a false sense of protection.
4. **DLQ for Kafka.** Failed click processing would route to a dead-letter topic instead of being dropped, and ops would monitor and replay it.
5. **Read replicas for analytics.** Aggregation queries would run against a replica, not the primary, so analytics load never affects redirect performance.
6. **Secrets management.** Credentials sourced from Vault or AWS Secrets Manager, not application YAML.
7. **API versioning strategy.** `/api/v1/` is already in place. I'd document a formal deprecation policy on top of it.
8. **Contract testing.** The frontend-backend interface would be tested with Pact or similar to prevent silent breakage.
9. **URL safety scanning, extended.** Already implemented: Claude classifies submitted URLs and fails open. Production would layer in a signature-based check like Google Safe Browsing alongside the semantic AI layer.
10. **Audit logging.** Every creation and deletion written to an immutable audit log.
11. **CI-enforced test-count regression gate.** This session's dead-test discovery (Risk 6 in `RISKS_AND_TRADEOFFS.md`) is a good argument for a CI check that fails the build if the number of executed tests drops between runs, not just if any test fails.

Final Thoughts

This project was a good exercise in making deliberate choices under time pressure. The hardest part wasn't writing the code, it was deciding what not to build. Authentication, CDN integration, distributed tracing are all the right things to

build eventually, but half-finished stubs would be worse than documenting them as known gaps.

The AI safety check is still the most interesting addition. Using Claude as a runtime component instead of only a coding assistant is a more honest demonstration of what "AI-assisted engineering" means end to end, since the model shows up in the architecture diagram itself, not just in the commit history.

One useful lesson along the way: the codebase's own test suite was quietly running less than it claimed to, and that's the kind of gap that only shows up when someone actually re-reads the build output instead of trusting a green checkmark. If I were to keep working on this, proper authentication is still the next thing I'd add. Everything else here is a scalability or reliability concern; that one's the security gap that matters most in a real deployment.